# Système d'exploitation : TP4, introduction aux sémaphores

février 2026 — Pierre Rousselin

**Exercice 1 : `million.c` : section critique**

1. Télécharger sur moodle le fichier `million.c` (vu en cours). Le lire, le compiler et exécuter le programme plusieurs fois pour vérifier que le résultat n'est pas déterminé.
2. On a vu en cours que le problème vient du fait que l'instruction `++incremente_moi;` n'est pas atomique. Il faudrait donc s'assurer qu'aucun *thread* ne commence cette instruction si un autre ne l'a pas terminée. On parle de *section critique*. En utilisant un sémaphore binaire (c'est-à-dire dont le compteur vaut toujours 0 ou 1), faire en sorte qu'il ne puisse jamais y avoir plus d'un *thread* dans cette section critique. Bien sûr, vérifier le résultat en exécutant plusieurs fois le programme.

   Pour utiliser le sémaphore, plusieurs stratégies sont possibles (vous pouvez en essayer plusieurs), par exemple
   - le sémaphore est global, donc visible dans tout le programme, initialisé (et détruit) dans `main` ;
   - le sémaphore est local à `main`, son adresse est passée en argument lors du `pthread_create`.

--- * ---

**Correction de l'exercice 1 :** Solution avec un sémaphore dont l'adresse est passée en argument.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define N 1000
void *bosser(void *);
int incremente_moi = 0;
int main()
{
        pthread_t th[N];
        sem_t s;
        sem_init (&s, 0, 1); /* un thread peut accéder à la section critique */
        int i;
        for (i = 0; i < N; ++i)
                pthread_create(&th[i], NULL, bosser, &s);
        for (i = 0; i < N; ++i)
                pthread_join(th[i], NULL);
        sem_destroy(&s);
        printf("incremente_moi = %d\n", incremente_moi);
        return 0;
}
void *bosser(void *s)
{
        int i;
        for (i = 0; i < N; ++i) {
                sem_wait(s);
                ++incremente_moi;
                sem_post(s);
        }
        return NULL;
}
```

--- * ---

**Exercice 2 : Le problème du rendez-vous (à 2)**

Le problème du rendez-vous apparaît lorsqu'on veut que deux tâches (ou plus) se synchronisent à un certain point de leur exécution (on parle aussi de barrière).

Pour montrer cette situation, on considère le programme `rendez_vous_a_2_non-sync.c` dans lequel deux *threads* se séparent la lourde tâche de compter de 0 à 100 en même temps sans s'écouter.

**1.** Télécharger ce programme sur moodle, le lire, le compiler et l'exécuter. Les nombres sont-ils dans l'ordre ?

**2.** On voudrait qu'il n'y ait pas de décalage et que les $a$ et $b$ donnent les nombres par paires (pas forcément dans le même ordre entre a et b), par exemple autoriser

```
a : 0
b : 0
b : 1
a : 1
```

mais pas

```
a : 0
a : 1
b : 0
b : 1
```

À l'aide de 2 sémaphores, obtenir ce résultat. Pour ce faire, on initialisera, dans la pile du *thread* initial, un tableau de 2 sémaphores dont l'adresse sera transmise aux deux autres *threads*.

--- * ---

**Correction de l'exercice 2 :**

**1.** Il n'y a aucun mécanisme de synchronisation, les deux *threads* n'ont aucune raison de prendre le même temps pour compter et les affichages finissent par être dans le désordre.

**2.** Deux sémaphores suffisent, chacun est à 1 lorsque le *thread* associé a terminé et à 0 sinon.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define COEFF_ATTENTE 100000
#define COEFF_DELTA   5000

void *compteur1(void *);
void *compteur2(void *);
void prendre_du_temps_pour_compter();

int main()
{
        pthread_t th1, th2;
        sem_t th_ok[2];

        sem_init(th_ok, 0, 0);
        sem_init(th_ok + 1, 0, 0);

        if (pthread_create(&th1, NULL, compteur1, th_ok) != 0) {
                perror("create th1");
                return 1;
        }
        if (pthread_create(&th2, NULL, compteur2, th_ok) != 0) {
                perror("create th2");
                return 1;
```

```
        }

        pthread_join(th1, NULL);
        pthread_join(th2, NULL);
        sem_destroy(th_ok);
        sem_destroy(th_ok + 1);
        return 0;
}

void *compteur1(void *sem_tab)
{
        sem_t *th_ok = sem_tab;
        int i;
        for (i = 0; i <= 100; i = i + 1) {
                prendre_du_temps_pour_compter();
                /* j'informe l'autre thread que j'ai fini */
                sem_post(th_ok);
                /* si besoin j'attends qu'il ait fini */
                sem_wait(th_ok + 1);
                printf("a : %d\n", i);
        }
        return NULL;
}

void *compteur2(void *sem_tab)
{
        sem_t *th_ok = sem_tab;
        int i;
        for (i = 0; i <= 100; i = i + 1) {
                prendre_du_temps_pour_compter();
                /* j'informe l'autre thread que j'ai fini */
                sem_post(th_ok + 1);
                /* si besoin j'attends qu'il ait fini */
                sem_wait(th_ok);
                printf("b : %d\n", i);
        }
        sem_post(th_ok + 1);
        return NULL;
}

void prendre_du_temps_pour_compter()
{
        usleep(COEFF_ATTENTE + (rand() % 10) * COEFF_DELTA );
}
```

--- * ---

**Exercice 3 : Précédence, d'après le partiel 1 2021 – 2022**

Un travail comporte 5 tâches : les tâches A, B, C, D et E. Les contraintes sont les suivantes :

— la tâche E ne peut commencer que si C et D sont terminées ;

— la tâche C ne peut commencer que si A et B sont terminées ;

— la tâche D ne peut commencer que si B est terminée.

**1.** Dessiner le diagramme de dépendance (ou encore de précédence) associé à cette situation.

**2.** Le programme `dependance.c`, à télécharger sur moodle, exécute ces tâches séquentiellement. En utilisant des *threads* et des sémaphores, séparer les différentes tâches de manière à ce que le programme prenne le moins de temps possible, tout en respectant les contraintes.

--- * ---

**Correction de l'exercice 3 :**

```
void *threadAC(void *arg);
void *threadBD(void *arg);
sem_t b_fini;

int main()
{
        pthread_t tAC, tBD;
        sem_init(&b_fini, 0, 0);
        if (pthread_create(&tAC, NULL, threadAC, NULL) != 0) {
                perror("pthread_create");
                return 1;
        }
        if (pthread_create(&tBD, NULL, threadBD, NULL) != 0) {
                perror("pthread_create");
                return 1;
        }

        pthread_join(tAC, NULL);
        pthread_join(tBD, NULL);
        tache_e();
        sem_destroy(&b_fini);
        return 0;
}
void *threadAC(void *arg)
{
        tache_a();
        sem_wait(&b_fini);
        tache_c();
        return NULL;
}
void *threadBD(void *arg)
{
        tache_b();
        sem_post(&b_fini);
        tache_d();
        return NULL;
}
```

--- * ---